

CS6135 VLSI Physical Design Automation

Homework 2: k-way Min-cut Partitioning

賈俊佑 studentID:114062645

1.Introduction

Implement and modify Fiduccia-Mattheyses(FM) algorithm to solve the problem of two-way/four-way min-cut partitioning.

2.Objective

The goal is to minimize the cut size ,the total number of nets that cross between partitions while strictly adhering to a balance constraint.

For each group in two-way min-cut partitioning:

$$0.45 \leq \frac{size(G)}{size(Total)} \leq 0.55$$

For each group in four-way min-cut partitioning:

$$0.225 \leq \frac{size(G)}{size(Total)} \leq 0.275$$

3.Result

```
+-----+
|      This script is used for PDA HW2 grading.      |
+-----+
host name: ic21
compiler version: g++ (GCC) 9.3.0

grading on 114062645:
checking item | status
+-----+
correct tar.gz | yes
correct file structure | yes
have README | yes
have Makefile | yes
correct make clean | yes
correct make | yes

testcase | #ways | cut size | runtime | status
+-----+
public1 | 2 | 308 | 1.53 | success
public1 | 4 | 1030 | 2.47 | success
public2 | 2 | 2152 | 41.98 | success
public2 | 4 | 5757 | 77.96 | success
+-----+

Successfully write grades to HW2_grade.csv
+-----+
```

4.Question

I. Where is the difference between your algorithm and FM Algorithm described in class?

The first difference is the balance constraint; in class the constraint of balance is:

$$rW - S_{\max} \leq |A| \leq rW + S_{\max}$$

r is the desired size ratio for a partition, W is the total size of all cell, and $S_{\max}$ of the single largest cell in the design.

But in this homework the balance constraint is:

$$0.45 \leq \frac{\text{size}(G)}{\text{size}(Total)} \leq 0.55$$

The second difference is the initial partitioning method. The method taught in class involves sorting cells by their size in ascending order and then adding them to the first partition until the balance constraint is satisfied. In contrast, the method I use first randomly shuffles the entire list of cells. Then, it adds each cell one-by-one to whichever partition is currently smaller by area.

The third difference is a modification to overcome its tendency to get trapped in local minima. I have introduced a perturbation step. When the algorithm detects that a pass has not to improve the cut size, instead of terminating, it calls a `perturb_partition` function. This function isolates small parts of cells and attempts to swap cells between the two partitions one pair at a time. For each potential swap one cell from partition A and one from partition B the algorithm first verifies that the exchange will not violate the overall partition balance constraints. This method of targeted, small-scale swaps effectively shakes up the solution by altering a localized area, allowing the algorithm to jump out of a local minimum and resume its search from a new state, thereby increasing the chance of discovering a better overall result.

II. Did you implement the bucket list data structure?

✓ If so, is it exactly the same as that described in the slide? How

many are they?

✓ If not, why? You had a better data structure? Or, is bucket list

useless?

The data structure I use is functionally the same as the one described in the slides. I implemented the bucket list using a vector<list<Cell *>>, where the vector's index directly corresponds to a cell's gain.

Since vector indices must be non-negative, but a cell's gain can range from $-P_{max}$ to $+P_{max}$, I map the gain to a valid index by adding an offset. Therefore, a cell with a gain is stored at the vector index $\text{gain} + P_{max}$. All cells sharing the same gain are then stored in the list at that specific index. This structure is highly efficient for finding the cell with the maximum gain and for maintaining the data structure throughout the process.

My implementation utilizes two such bucket lists, one for each partition. This separation is necessary to independently track the best cell to move from each side.

III. How did you find the maximum partial sum and restore the result?

1. Finding the Best State

Instead of explicitly calculating the maximum partial sum of gains, my algorithm finds the optimal state by directly tracking the minimum cut size achieved during a single pass. The goal is to find the prefix of the move sequence that produces the largest total gain, which is the same as finding the point with the lowest cut size.

During a pass, the algorithm maintains a running cutsize. After each cell is moved, this cutsize is updated by subtracting the gain of that cell. To track the best state, three key variables and array(vector) are used:

min_cutsizes: Stores the lowest cut size seen in the current pass.

best_iteration: Stores the step number at which the min_cutsizes occurred.

order: A vector that records a pointer to every cell that is moved, in the exact order they are moved.

After each move, the algorithm checks if the current cutsizes is less than `min_cutsizes`. If it is, `min_cutsizes` is updated, and the current move number is saved to `best_iteration`. By the end of the pass, `best_iteration` holds the exact number of moves that leads to the best result.

2. Restoring the Partition to the Best State

Once the pass is complete and the optimal point is identified by `best_iteration`, the partition must be reverted to that exact state.

This process uses the order vector, which contains the complete history of all moves made. The algorithm iterates backwards from the very last move in the order vector down to the move immediately after the one recorded in `best_iteration`. For each cell in this range, its move is simply reverted by assigning it back to its original partition.

By the way, in addition to finding the best state within a single pass, my algorithm also maintains a global best state across all passes. I use a vector to store the partition assignment of every cell corresponding to the best cut size found throughout the entire run. This is critical because my implementation uses a perturbation step to escape local minima. Finally, after all passes are complete, the algorithm restores this global best state to guarantee the final output is the best solution found.

IV. How did you modify the FM algorithm to handle four-way min-cut partitioning? You could use flow chart(s) and/or pseudo code to help elaborate your algorithm.

To handle four-way partitioning, I implemented a Recursive Bisection strategy, which reuses the two-way FM partitioner in a hierarchical process. First, the entire circuit is split into two large intermediate groups, and then each of these groups is partitioned independently to produce the four final partitions. The key to this approach is managing the hierarchical size constraints. While the final goal requires each of the four partitions to have a size between 0.225 and 0.275 of the total, this dictates that the first bisection must produce intermediate groups that are themselves valid for further subdivision. To guarantee this and prevent a potential deadlock, I specifically constrained the first split so that each of the two intermediate groups must have a size between 0.47 and 0.53 of the total. This ensures that both resulting groups are

balanced enough to be successfully split again while adhering to the final size requirements.

The algorithm step:

1.Top-Level Bisection (First Split):

The `two_way_partitioning` function is called on the entire set of cells (`cell_array`) and nets (`net_array`).The circuit is divided into two large partitions, temporarily labeled Partition 0 and Partition 1.

2.Sub-Problem Creation:

The cells are segregated into two new groups, `cell_array0` and `cell_array1`, based on the result of the first split.

Crucially, two new netlists, `net_array0` and `net_array1`, are created. `net_array0` contains only the nets that are entirely internal to `cell_array0`, and `net_array1` contains nets internal to `cell_array1`. This isolates the two sub-problems so they can be partitioned independently.

3.Second-Level Bisections (Second and Third Splits):

The `two_way_partitioning` function is called again, once for each sub-problem:

```
two_way_partitioning(cell_array0, net_array0)
```

```
two_way_partitioning(cell_array1, net_array1)
```

`cell_array0` is split into its own partitions 0 and 1. These become our final partitions 0 and 1. `cell_array1` is also split into its own partitions 0 and 1.

4.Partition Remapping:

Since the second partition of `cell_array1` also produced partitions labeled 0 and 1, they must be remapped to unique final values to avoid collision. The partitions from `cell_array1` are re-assigned: temporary 0 becomes final partition 2, and temporary 1 becomes final partition 3.

5.Final Calculation:

After all cells have been assigned to one of the four final partitions (0, 1, 2, or 3), the total four-way cutsize is calculated across the original, complete netlist.

The pseudo code:

```

FUNCTION four_way_partition (all_cells, all_nets):
    // STEP 1: Perform the first bisection on the entire circuit.
    // This assigns each cell in all_cells to partition 0 or 1.
    two_way_partition(all_cells, all_nets)

    // STEP 2: Create two sub-problems based on the first split.
    cells_A = {all cells with partition == 0}
    cells_B = {all cells with partition == 1}

    // Isolate nets that are internal to each partition.
    nets_A = {nets where all connected cells are in cells_A}
    nets_B = {nets where all connected cells are in cells_B}

    // STEP 3: Partition each sub-problem independently.
    // The results overwrite the partition numbers within each subset.
    two_way_partition(cells_A, nets_A) // Partitions cells_A into 0 and 1
    two_way_partition(cells_B, nets_B) // Partitions cells_B into 0 and 1

    // STEP 4: Remap the partitions from the second group to their final
    values.
    FOR EACH cell IN cells_B:
        IF cell.partition == 0 THEN
            cell.partition = 2
        ELSE
            cell.partition = 3
        END IF
    END FOR

    // STEP 5: Calculate the final cutsizes.
    final_cutsizes = calculate_four_way_cutsizes(all_nets)
    RETURN final_cutsizes

```

5. Conclusion: Learnings and Challenges

For this homework, I found that implementing the pseudocode from the slides in C++ wasn't too difficult. The real challenge was the strict time constraint, which required the FM algorithm to run in under 100

seconds.

Initially, my implementation relied heavily on the map data structure. I used it for the cell and net arrays, and also for the bucket list (as `map<int, list<Cell*>>`), because it was easy to work with. However, this approach was too slow; the overhead from maintaining the maps caused the program to take over 1500 seconds to execute.

To solve this performance issue, I switched all these data structures to vector. Since a vector uses a contiguous block of memory, it offers much faster access and manipulation. This key change refactoring not only the cell and net arrays but also the bucket list from a map to a `vector<list<Cell*>>`—was what allowed my code to meet the time limit.

From this homework, I learned first how critical choosing the correct data structure is for performance. My initial implementation using map was easy to write but was far too slow, failing to meet the time constraint. Switching to vector provided a dramatic speed-up and was the key to solving the performance bottleneck. This was a powerful practical lesson on how theoretical time complexity impacts execution time.

Secondly, I'm learning from implementing this project is the inherent sensitivity of the Fiduccia-Mattheyses (FM) algorithm to its initial partition. I observed that the quality of the final min-cut solution is heavily dependent on the randomly generated initial partition. This is because the FM algorithm is a greedy, heuristic-based approach; it iteratively improves upon a solution to find a local minimum, which is not guaranteed to be the global minimum.

6. Declaration on the Use of AI Tools

In this project, I used AI tools including Google's Gemini and OpenAI's ChatGPT, for two primary purposes:

1. Debugging: I submitted portions of my code to these tools to help identify logical errors, syntax issues, and potential bugs that were difficult to trace manually.

2.Code Improvement and Optimization: I consulted the AI models for suggestions on how to enhance the performance of my algorithm. This included asking for recommendations on more efficient data structures and alternative implementation strategies that could accelerate the program's execution speed.